COMP-8117 · SPLASH PROPOSAL · DRAFT FOR TEAM

Pay-per-use AI hosting for a real team

We help a real team run their AI model so they stop paying for idle GPUs, without slow or failed responses when traffic spikes.

The problem

> [Real team] pays for a GPU that sits idle most of the day. Turning it off saves money but makes users wait or get errors when usage suddenly jumps. **How can they pay only for what they use and still stay fast and reliable?**

We prove the problem is real with **their** evidence: their GPU bill, their usage logs, past slowdowns or errors, and an interview with whoever pays or fixes it.

Who is involved

- **Feels the problem:** the people using the model (they wait or get errors)
- **Owns the problem:** whoever pays the GPU bill (lab lead, founder)
- **Can change it:** the team's developer or admin
- **Uses our software:** that developer, who deploys the model on our setup

What we research

1. Where does the money go? Busy hours vs idle hours.
2. What does usage look like? Steady, bursty, deadline spikes?
3. How long can users wait before it's a problem?
4. Which option fits best? We compare smaller GPU, smaller (quantized) model, serverless scale-to-zero, spot instances, batching, and a queue.
5. We test the top 2–3 options with their real model and measure the trade-offs.

The finding is which option wins for *their* usage, backed by numbers. We don't pick the answer first.

What we build

A small system, only as much as the findings need: an API that accepts requests, a queue that absorbs spikes, GPU workers that scale down to zero when idle, and a dashboard showing cost, wait time and errors. We deploy it through dev, staging and prod.

No user accounts, no polished frontend, no business model.

How we know it worked

We set targets **before** building, then run their real workload on our setup for 2–3 weeks and compare it with their old setup.

MEASURE	TARGET	EVIDENCE
Monthly GPU cost	≥ 50% lower	Their bill, before vs after
Cost when idle	≈ $0	Worker count 0 + billing
Failed requests	≈ 0%	Logs and dashboard
Response time (95%)	≤ X s	Dashboard; X agreed with the team

Exact targets are set once we see their data.

Roles

No one needs DevOps experience except the Platform lead. Deployment is one command with a written guide, and everyone pairs on at least one deploy.

ROLE	OWNER	DOES
Platform / DevOps (lead)	Abhinav	Cloud, deploys, scaling, budget
Research & stakeholder	[name]	Interviews, bill and usage analysis, success targets
Backend	[name]	API and queue (regular Python coding)
Testing & evaluation	[name]	Load tests, before/after comparison, rubrics

How this earns marks

GRADED ON	HOW WE COVER IT
Real, documented problem	Real team, their bill, logs and interview
Intervention follows from analysis	Options compared with data before we build
Product works (15%)	Runs on their real model and traffic
Method and resources (15%)	Decisions log, budget tracking incl. our hours, dev → staging → prod
Problem reduced	Before/after numbers against preset targets
Scalable impact	Plan for bigger models and more teams
Judgment	Our own rubrics, compared with the teaching team's
Reflection (10%, individual)	Each person writes about their own decisions at every milestone

Timeline

WHEN	WHAT
`Wk 3 (now)`	Expert Review 1: present the pivot to a real team
`Wk 4`	Find and sign the real team (Reading Week)
`Wk 5–6`	Collect their data, compare options, set targets. Expert Review 2 (Oct 28)
`Wk 7–8`	Build and deploy the chosen setup; run on staging
`Wk 9–10`	Real usage for 2–3 weeks, measure before/after (ends Nov 25)
`Wk 12`	Final defense: live demo plus evidence

Budget

About **$250–$640** total, using cloud credits first. Hard cap of $1,000/month with alerts and a kill switch. The real team's own spend should go *down*.

Main risk

We can't find a real team by the end of week 4. Fallback: the same approach on cloud waste in a real account (a lab's or a small company's), which is cheaper and simpler and still measured with real bills.

Why this project

- Real problem with hard numbers, so it's easy to show results
- Clear roles where everyone has their own work to show
- Strong portfolio piece: cloud, AI and cost engineering